

用 Triton 写 GPU 算子

从 AI Infra 到训练对照

Version 0.1

ai_infra

2026 年 9 月

目录

前言	3
1 从 AI Infra 到 Triton	4
1.1 AI Infra 是什么	4
1.2 算子为什么重要	5
1.3 GPU 长什么样	5
1.4 CUDA 的编程模型	6
1.5 Triton 在 CUDA 上做了哪些抽象	6
2 Triton 语法	7
2.1 心智模型	8
2.2 最小完整程序	8
2.3 Kernel 里能写什么	9
2.4 常用原语	10
2.5 常见坑	10
3 向量加法	10
3.1 公式与数据流	10
3.2 怎么跑	11
3.3 正确性	11
3.4 效果	12
4 激活: ReLU 与 SwiGLU	12
4.1 公式与数据流	12
4.2 怎么跑	13
4.3 正确性	14
4.4 效果	14
5 矩阵乘	14

5.1	公式与数据流	14
5.2	怎么跑	16
5.3	正确性	16
5.4	效果	17
6	Softmax	17
6.1	公式与数据流	17
6.2	怎么跑	20
6.3	正确性	20
6.4	效果	21
7	LayerNorm 与 RMSNorm	21
7.1	公式与数据流	21
7.2	怎么跑	22
7.3	正确性	22
7.4	效果	22
8	注意力: 从 Naive 到 Flash	23
8.1	Naive 与多头: Host 串起来不算融合	23
8.2	单头 Flash: 论文双循环	23
8.3	教学多头 causal	26
8.4	加速版: 训练在用的核	27
8.5	RoPE、GQA、MLA	29
8.6	怎么跑	29
8.7	正确性	29
8.8	效果	30
9	用 nsys 看 Launch	30
9.1	看什么	30
9.2	怎么跑	30
9.3	正确性	31
9.4	效果	31
10	推理实战: 接到 Qwen3.5	31
10.1	公式与数据流	31
10.2	怎么跑	32
10.3	正确性	32

10.4 效果	33
11 训练对照: Function 与三后端	33
11.1 公式与数据流	33
11.2 怎么跑	34
11.3 正确性	35
11.4 效果	35
A 容易踩的坑	35
B 符号与文件	36

前言

本讲要点

- 读者: 会一点 Python / PyTorch
- 不需要 CUDA C++
- 用解释器跑 .py
- 环境: WSL2 + CUDA 12.8
- 算子章四块骨架
- 教学核常慢于库函数

这份讲义跟着仓库走: 先写 kernel, 再接到 generate 和若干步训练上。读者需要会一点 Python 和 PyTorch, 不需要会 CUDA C++。

怎么用这本书

正文分四部分。第一部分回答「为什么是 Triton」; 第二部分按课号写算子; 第三、四部分把核接到推理和训练。每一章算子尽量按同一骨架写: 公式与数据流、怎么跑、正确性、效果。算子优化往往要靠数据流图才能看懂: 先看图里哪一维用 pid、哪一维必须核内 for、tile 住 HBM 还是 SRAM, 再对照摘录的代码。

Note

效果一节会反复强调: 教学核常常慢于 cuBLAS / SDPA。课看的是分块、融合和 launch, 不是端到端赢库函数。

完整程序在仓库里, 书中只摘关键片段。路径相对于仓库根目录, 例如

```
kernel/triton/tutorial/01_vector_add/step01_add_1d.py
```

启动方式

跑示例请用解释器，不要把 `.py` 当 shell 脚本执行：

```
cd /path/to/ai_infra
source .venv/bin/activate
python kernel/triton/tutorial/01_vector_add/step01_add_1d.py
```

环境

参考配置：RTX 5070 Laptop + WSL2。Blackwell (sm_120) 需要 **CUDA 12.8** 的 PyTorch 轮子。

本仓库环境

```
bash scripts/setup_env.sh
source .venv/bin/activate
```

Remark

Windows 原生对 Triton 支持很弱，建议 Linux 或 WSL2。本机 `torch.profiler` 常常采不到 GPU，看 `launch` 用第 9 章的 `nsys`。作者进度板 `kernel/triton/tutorial/学习计划书.md` 不是教材，附录只提炼其中仍然有效的坑。

1 从 AI Infra 到 Triton

本讲要点

- 训练 / 推理栈怎么分层
- 算子改的是 HBM 与 `launch`
- GPU: HBM / SM / warp
- CUDA: grid / block / thread
- Triton 在 CUDA 上抬了一层
- Triton 不替代 cuBLAS

写 kernel 之前，先看它落在训练/推理栈的哪一层，以及 GPU / CUDA 把并行切成哪几级。这一章几乎没有代码：把硬件、CUDA 的位置和 Triton 收掉了哪些细节说清楚，下一章再写 `pid` 与 `mask`。

1.1 AI Infra 是什么

大模型能跑起来，靠的不只是「Transformer 公式」。从用户请求到 GPU 上的一条指令，中间至少叠了这些层：

1. **框架**：PyTorch 描述计算图、自动微分、混合精度。

2. **编译器**: `torch.compile` / Inductor 把图收成更少的 kernel。
3. **算子库**: cuBLAS、cuDNN、FlashAttention 等, 把 GEMM、卷积、注意力做成高度特化的实现。
4. **运行时与调度**: vLLM 这类系统做 PagedAttention、continuous batching、CUDA Graph, 决定**何时** launch、**launch 哪一批**请求。
5. **硬件**: HBM、SRAM、Tensor Core、SM 数量。同一套公式, 访存路径不同, 墙钟差一个数量级并不罕见。

本仓库落在哪

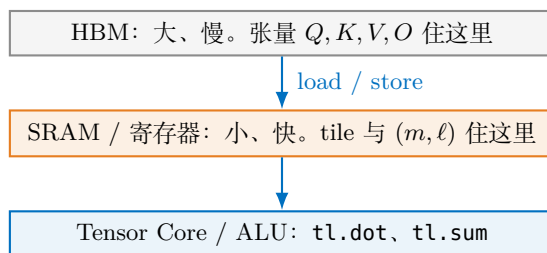
我们写的是第 3 层附近: **自己实现若干算子**, 并接到一条 generate 和若干步训练上。不是造一个迷你 vLLM, 也不追求端到端打败库函数。

1.2 算子为什么重要

同一套 Llama / Qwen 式结构里, 时间往往耗在三类事情上:

- 大矩阵乘 (线性层、SwiGLU 的 gate/up/down);
- 归一化 (LayerNorm / RMSNorm);
- 注意力 ($QK^\top$ 、softmax、 PV , 以及 causal / GQA 变体)。

换核改变的是这一层的 **HBM 往返**、**片上 SRAM**、**launch 次数**, 不是模型公式本身。

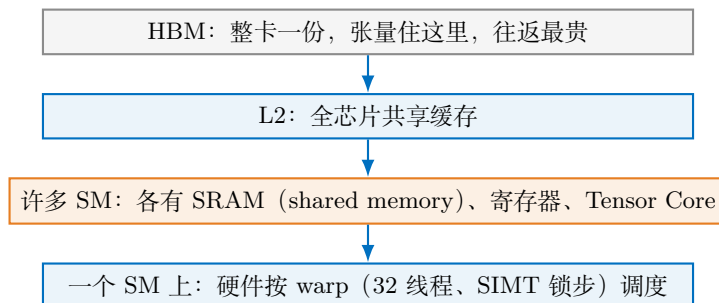


换核改的是 HBM 往返和片上驻留, 不是 Transformer 公式。

教学核如果比 cuBLAS 慢, 通常是 GEMM 形状已经够大、库函数吃满了 Tensor Core; 这不说明「不该学分块」, 只说明**对照对象要选对**。

1.3 GPU 长什么样

NVIDIA GPU 可以先看成两层: 外面一份大而慢的内存, 里面许多能并行干活的小核心。



一块 GPU = 慢而大的 HBM + 许多能独立干活的 SM。算子优化就是少碰 HBM、让 tile 留在 SM 上。

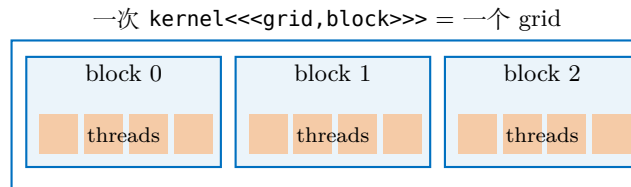
- **HBM** (全局内存): 容量大 (十几到几十 GB), 带宽相对片上仍慢。 Q, K, V, O 以及中间的 S, P 默认住这里。
- **L2**: 全芯片共享的缓存, 课上不用手管。
- **SM** (Streaming Multiprocessor): 真正算的地方。每个 SM 有自己的 **shared memory** (SRAM, 大约一百 KB 量级)、寄存器和 Tensor Core。一个 CUDA **block** 调度到其中一个 SM, 块内线程才能共用这份 SRAM。
- **warp**: 硬件调度单位, **32 条线程锁步** (SIMT)。Tensor Core 吃的是 warp 级的矩阵乘; 一条线程跑到 if、另一条没跑到, 就是 warp divergence。

和算子的关系

Flash 把 S, P 留在 SRAM, 是因为 HBM 往返比多算几次 softmax 统计量更贵。教学核 $D = 512$ 跑不了, 是因为这一份 tile 超过了单 SM 的 shared memory, 不是公式错了。

1.4 CUDA 的编程模型

CUDA 把上面的硬件收成 Host 能启动的三级并行:



每个 block 落到一个 SM, 块内线程共享 SRAM。CUDA 要你写「第 t 号线程做什么」。

- **grid**: 一次 launch 的全部工作。Host 写 `kernel<<<grid, block>>>`。
- **block**: 一份驻留在某个 SM 上的线程组, 共享 SRAM, 用 `__syncthreads` 对齐。
- **thread**: 你写的那一层。地址通常是 `blockIdx * blockDim + threadIdx`。

CUDA C++ 仍然是 NVIDIA GPU 上性能的底盘: warp、shared memory、occupancy、异步拷贝都可以直接管; cuBLAS / 官方 Flash 最终也是这一层。缺点是入门必须同时处理 thread 索引、同步、bank conflict 和 divergence。本课不写 CUDA C++, 但后面的 `pid`、`BLOCK`、`num_warps` 都是对着这张图说的。

1.5 Triton 在 CUDA 上做了哪些抽象

Triton 编译器的后端仍是 CUDA (PTX / cubin, 同一套驱动)。它抬高的是你要写到哪一层。



编译器把 tile 摊到 warp。grid 和怎么分块仍要你定。

表 1: Triton 收掉了什么、留下了什么

	CUDA C++	Triton
并行单元	thread / warp	program $\approx$ block
区分实例	threadIdx / blockIdx	tl.program_id
切一块数据	用线程号算偏移	BLOCK+arange+mask
块内同步 / SRAM	__shared__、__syncthreads	编译器插入
喂 Tensor Core	自己排 warp / MMA	tl.dot
几个 warp	blockDim	num_warps (常 autotune)
异步拷贝	自己写	num_stages
启动	<<<grid,block>>>	kernel[grid](...)

收掉 thread，不收掉分块

Triton 不替你决定哪一维用 pid、哪一维核内 for，也不替你决定中间结果住 HBM 还是 SRAM。融合、causal、GQA、launch 次数仍然是数据流问题。它收掉的是「第 t 号线程怎么对齐、这块 tile 怎么摊到 4 个 warp 上」。

所以：会 Triton 不是「不懂 GPU」。HBM / SM / warp 这张骨架还在；只是你从 tile 往下看，不再从 thread 往上看。

Triton 不替代 CUDA

教学核常常慢于 cuBLAS 和 SDPA。该调库函数的地方（大 GEMM、官方 Flash）课上仍调库。Triton 的价值是：用可改的数据流把融合、causal、GQA、反向讲清楚，并在需要时替换个别算子。

对照如下。

表 2: 三种写法各自管到哪一层

	PyTorch eager	Triton	CUDA C++
并行单元	算子调用	program / 块	thread / warp
典型作者	模型代码	算子数据流	硬件细节
和 autograd	自动	需 Function	需 Function + 更多样板
性能上限	依赖后端选择	中高（看形状）	最高
本课用法	对照基线	主体	不写

下一章从最小的向量加法开始，把 pid、BLOCK、mask、stride 写成可跑的套路。

2 Triton 语法

本讲要点

- program / pid / BLOCK / mask
- 最小 add_kernel
- stride 与二维广播

- `tl.dot`、归约、`atomic_add`
- 常见坑

Triton 用类 Python 语法写 GPU kernel。第 1 章已经把 program 对上 CUDA block：你不写「第 t 号线程处理一个元素」，而是：

一块数据一份 program

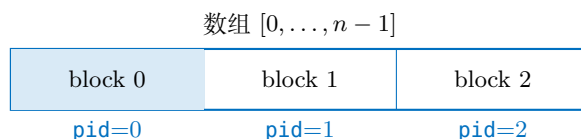
每个 **program** ($\approx$ 一个 CUDA block，落到一个 SM) 一次处理一整块数据 (`BLOCK_SIZE` 个元素)。块内的 thread / warp 由编译器摊，`num_warps` 只是提示。

2.1 心智模型

表 3: 四个天天出现的词

概念	含义
program / pid	并行执行的一份 kernel 实例， <code>tl.program_id</code> 区分
<code>BLOCK_SIZE</code>	每个 program 一次处理多少元素，通常是 2 的幂
pointer + offset	指针算术定位，再 <code>tl.load</code> / <code>tl.store</code>
mask	长度不是块整数倍时，多余位置不读不写

数据被切成多块，每个 pid 负责一块：



一份 program 吃一块 BLOCK；尾巴用 mask。

2.2 最小完整程序

套路只有四步：`pid+BLOCK` 得到 `offsets`；`mask`；`load` $\rightarrow$ 算 $\rightarrow$ `store`；Host 算 `grid` 并启动。仓库里可跑的版本如下（已挂 autotune，Host 不再传 `BLOCK_SIZE`）。

```
@triton.jit
def add_kernel(
    x_ptr,
    y_ptr,
    out_ptr,
    n_elements,
    BLOCK_SIZE: tl.constexpr,
):
    # 当前 program 编号
    pid = tl.program_id(axis=0)
    # 本 program 负责的全局下标
    offsets = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)
    # 边界保护
```

```

mask = offsets < n_elements

x = tl.load(x_ptr + offsets, mask=mask)
y = tl.load(y_ptr + offsets, mask=mask)
tl.store(out_ptr + offsets, x + y, mask=mask)

def add(x: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
    assert x.is_cuda and y.is_cuda
    assert x.numel() == y.numel()
    out = torch.empty_like(x)
    n = x.numel()
    grid = lambda meta: (triton.cdiv(n, meta["BLOCK_SIZE"]),)
    add_kernel[grid](x, y, out, n)
    return out

```

Listing 1: 向量加法: kernel 与 Host 启动

文件: kernel/triton/tutorial/01_vector_add/step01_add_1d.py:21--48

Note

kernel[grid](...) 按 grid 启动。triton.cdiv(n, B) 是向上取整。有 @triton.autotune 时不要再手传 BLOCK_*, grid 用 lambda meta。

2.3 Kernel 里能写什么

- @triton.jit 编译成 GPU kernel。里面只能用 tl.* 和受限 Python 子集，**不能调 NumPy / PyTorch**。
- 指针由 CUDA tensor 传入；长度等运行时标量每次可改；BLOCK_SIZE: tl.constexpr 影响代码生成。
- 向量级选择用 tl.where，不要对每个元素写 Python if。编译期循环可用 tl.static_range；Triton 里没有 break。

二维 tile 的索引几乎总是广播：

```

offs_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
offs_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
offs = offs_m[:, None] * stride_m + offs_n[None, :] * stride_n
mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)

```

stride 是「逻辑维 +1 时内存跳几格」。地址 = $\text{base} + \sum \text{offs}_i \cdot \text{stride}_i$ 。不要假设矩阵一定连续。

	(0,0)	(0,1)	(0,2)
M	(1,0)	(1,1)	(1,2)
	(2,0)	(2,1)	(2,2)

N

二维: pid_m 管行块, pid_n 管列块, 各自乘 stride。

2.4 常用原语

累加器常用 FP32, 即使输入是 BF16。矩阵乘的核心是 `tl.dot`。归约是 `tl.sum / tl.max` (Softmax、Norm、Attention 都会用)。多 program 往同一地址加, 才用 `tl.atomic_add`, 并且 autotune 必须 `reset_to_zero`。Host 侧计时用 `triton.testing.do_bench`; 对照前先 `torch.cuda.synchronize`, GEMM 对照请关掉 TF32。

2.5 常见坑

1. 忘了 `mask`: 越界读写。
2. CPU tensor: 必须 `device="cuda"`。
3. 在 kernel 里用 PyTorch。
4. FP16 累加不稳: 归约用 FP32。
5. 把 Python 动态 `list` 搬进 kernel。

Problem 建议顺序

先向量加减 / ReLU, 再行 Softmax, 再小 matmul, 再 fused bias+activation。官方教程: <https://triton-lang.org/main/getting-started/tutorials/index.html>。

3 向量加法

本讲要点

- 一维 `mask`
- 二维 / 三维 `stride`
- 带宽绑定
- 文件: `01_vector_add/`

配套: `kernel/triton/tutorial/01_vector_add/`。先跑: `step01_add_1d.py` → `step02_add_2d.py` → `step03_add_3d.py`。

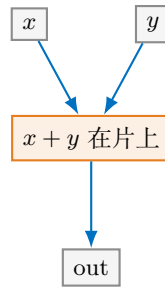
3.1 公式与数据流

$$\text{out} = x + y.$$

每个 program 处理一块 **BLOCK** 元素; 越界用 `mask`。二维、三维不要假设内存连续, 用各自的 `stride`。二维 kernel 把 `pid` 拆成行块和列块:

	(0,0)	(0,1)	(0,2)
M	(1,0)	(1,1)	(1,2)
	(2,0)	(2,1)	(2,2)
	N		

二维: `pid_m` 管行块, `pid_n` 管列块, 各自乘 `stride`。



带宽绑定：一次加法，时间看两次 load、一次 store。

```

pid_m = tl.program_id(axis=0)
pid_n = tl.program_id(axis=1)

offs_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
offs_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)

# 广播成 [BLOCK_M, BLOCK_N]; 各自用自己的 stride (转置/非连续时可能不同)
offs_x = offs_m[:, None] * stride_xm + offs_n[None, :] * stride_xn
offs_y = offs_m[:, None] * stride_ym + offs_n[None, :] * stride_yn
offs_o = offs_m[:, None] * stride_om + offs_n[None, :] * stride_on
mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)

x = tl.load(x_ptr + offs_x, mask=mask)
y = tl.load(y_ptr + offs_y, mask=mask)
tl.store(out_ptr + offs_o, x + y, mask=mask)
  
```

Listing 2: 二维加法：stride 与二维 mask

文件：kernel/triton/tutorial/01_vector_add/step02_add_2d.py:33--47

三维同理，再加一个 pid 轴。这是 launch 套路的操练场，不是算力测试。

3.2 怎么跑

怎么跑

```

python kernel/triton/tutorial/01_vector_add/step01_add_1d.py
python kernel/triton/tutorial/01_vector_add/step02_add_2d.py
python kernel/triton/tutorial/01_vector_add/step03_add_3d.py
  
```

3.3 正确性

Note

对照 $x + y$ 。elementwise 应对齐到 $\text{atol } 10^{-5}$ 量级。

3.4 效果

Remark

这是**带宽绑定**：算的是一次加法，时间看 load/store。autotune 主要扫 BLOCK / num_warps。不要拿它和 cuBLAS 比。

4 激活：ReLU 与 SwiGLU

本讲要点

- ReLU 前向 / 反向
- 浅 SwiGLU：HBM 上的 a, b
- 深融合：两次 `tl.dot`
- 反向重算 + cuBLAS GEMM

配套：kernel/triton/tutorial/02_activations/。先跑：step01_relu.py → step02_swiglu.py → step03_swiglu_fused.py
ReLU 只需第 2 章；深融合 SwiGLU 建议先扫一眼第 5 章 matmul（两次 `tl.dot`）。

4.1 公式与数据流

ReLU：

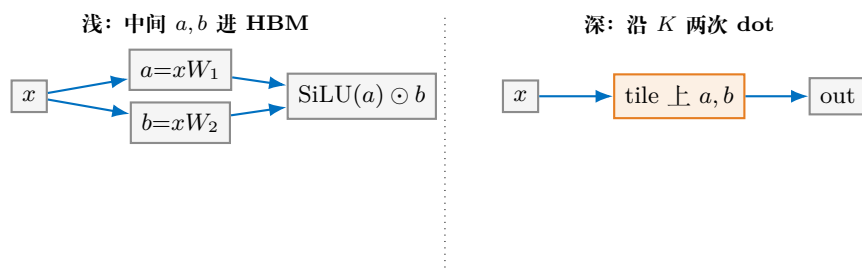
$$y = \max(x, 0), \quad dx = dy \cdot \mathbf{1}_{x>0}.$$

`tl.where(x > 0, x, 0.0)` 即可。反向在同文件 TritonReLU。

SwiGLU：

$$\text{out} = \text{SiLU}(xW_1) \odot (xW_2), \quad \text{SiLU}(a) = a \cdot \sigma(a).$$

- 浅** (step02_swiglu.py)：Host 上做两次 GEMM，kernel 只做 `silu(a)*b`，中间 a, b 写 HBM。
- 深** (step03_swiglu_fused.py)：同一 kernel 沿 K 做两次 `tl.dot`，SiLU 与逐元乘之后写回，不物化 a, b 。



深融合少两次中间 GEMM 的 HBM 往返；大矩阵仍可能慢于 cuBLAS。

```
acc1 = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32) # x @ W1
acc2 = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32) # x @ W2

for k in range(0, tl.cdiv(K, BLOCK_K)):
```

```

k_offs = k * BLOCK_K + offs_k

x_ptrs = x_batch + (
    offs_m[:, None] * stride_xm + k_offs[None, :] * stride_xk
)
w1_ptrs = w1_ptr + (
    k_offs[:, None] * stride_w1k + offs_n[None, :] * stride_w1n
)
w2_ptrs = w2_ptr + (
    k_offs[:, None] * stride_w2k + offs_n[None, :] * stride_w2n
)

x_mask = (offs_m[:, None] < M) & (k_offs[None, :] < K)
w_mask = (k_offs[:, None] < K) & (offs_n[None, :] < N)

x_tile = tl.load(x_ptrs, mask=x_mask, other=0.0)
w1_tile = tl.load(w1_ptrs, mask=w_mask, other=0.0)
w2_tile = tl.load(w2_ptrs, mask=w_mask, other=0.0)

# 同一份 x_tile 复用两次, 少读一次 x
# ieee: 便于和 PyTorch FP32 对齐; 追求速度可改 "tf32"
acc1 += tl.dot(x_tile, w1_tile, input_precision="ieee")
acc2 += tl.dot(x_tile, w2_tile, input_precision="ieee")

out = acc1 * tl.sigmoid(acc1) * acc2

o_ptrs = o_batch + (
    offs_m[:, None] * stride_om + offs_n[None, :] * stride_on
)
o_mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)
tl.store(o_ptrs, out, mask=o_mask)

```

Listing 3: 融合 SwiGLU: 沿 K 两次 dot, 再 SiLU 与乘

文件: kernel/triton/tutorial/02_activations/step03_swiglu_fused.py:80--114

TritonSwiGLU 的反向

正向是融合核; 反向重算 a, b 后, dW 仍走 PyTorch GEMM。这是教学接法, 不是再写一个 TritonLinear。

4.2 怎么跑

怎么跑

```

python kernel/triton/tutorial/02_activations/step01_relu.py
python kernel/triton/tutorial/02_activations/step02_swiglu.py
python kernel/triton/tutorial/02_activations/step03_swiglu_fused.py

```

4.3 正确性

Note

ReLU / TritonReLU 对 `F.relu` 与 `autograd.grad`。SwiGLU 对 `silu(x@W1)*(x@W2)`；fp32 建议关 TF32, `rtol = 10-3`。TritonSwiGLU 的 `dx, dW1, dW2` 对同一公式的 `autograd.grad`。

4.4 效果

Remark

深融合快在少两次中间 GEMM 的 HBM 往返。用 `do_bench` 看 step02 vs step03。大矩阵上教学融合仍可能慢于 cuBLAS 两次 GEMM 再 elementwise——第 11 章训练默认因此不换 SwiGLU。

5 矩阵乘

本讲要点

- 输出块沿 K 累加
- `reset_to_zero`
- 三维一次 launch
- step90 未收完，跳过

配套: `kernel/triton/tutorial/03_matmul/`。先跑: `step01_dot_1d.py` → `step02_dot_3d.py` → `step03_matmul_2d.py` → `step04_matmul_3d.py`。

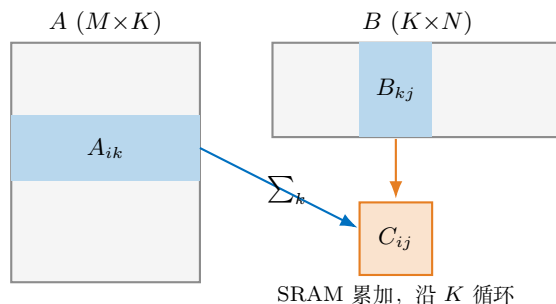
Note

选修 `step90_linear_backward.py` 未收完，初学者跳过。第 8 章 naive attention 会 import 三维 matmul。

5.1 公式与数据流

$$C = A @ B.$$

每个 program 负责输出块 (`BLOCK_M, BLOCK_N`)，沿 K 循环 `tl.dot` 累加。三维用 `pid_b` 一次 launch，不要 Python for `b`。



一个 program 只写一块 C ；归约维 K 必须在核内 for，不能拆成不同 pid。

```
pid_m = tl.program_id(axis=0)
pid_n = tl.program_id(axis=1)

offs_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
offs_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
offs_k = tl.arange(0, BLOCK_K)

# 累加器
acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)

# 沿 K 维分块累加
for k in range(0, tl.cdiv(K, BLOCK_K)):
    k_offs = k * BLOCK_K + offs_k
    a_ptrs = a_ptr + (offs_m[:, None] * stride_am + k_offs[None, :] * stride_ak)
    b_ptrs = b_ptr + (k_offs[:, None] * stride_bk + offs_n[None, :] * stride_bn)

    a_mask = (offs_m[:, None] < M) & (k_offs[None, :] < K)
    b_mask = (k_offs[:, None] < K) & (offs_n[None, :] < N)

    a = tl.load(a_ptrs, mask=a_mask, other=0.0)
    b = tl.load(b_ptrs, mask=b_mask, other=0.0)
    acc += tl.dot(a, b)

c_ptrs = c_ptr + (offs_m[:, None] * stride_cm + offs_n[None, :] * stride_cn)
c_mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)
tl.store(c_ptrs, acc, mask=c_mask)
```

Listing 4: 二维 matmul: 输出块沿 K 累加

文件: kernel/triton/tutorial/03_matmul/step03_matmul_2d.py:39--64

一维点积把多块部分和 `atomic_add` 到标量。autotune 若对同一输出累加，必须 `reset_to_zero`，否则 benchmark 污染结果：

```
@triton.autotune(
    configs=[
        triton.Config({"BLOCK_SIZE": 128}, num_warps=4),
        triton.Config({"BLOCK_SIZE": 256}, num_warps=4),
        triton.Config({"BLOCK_SIZE": 512}, num_warps=8),
        triton.Config({"BLOCK_SIZE": 1024}, num_warps=8),
```

```

],
key=["n_elements"],
# atomic_add 累加到同一标量; benchmark 前必须清零, 否则结果被污染
reset_to_zero=["out_ptr"],
)
@triton.jit
def dot_kernel(
    x_ptr,
    y_ptr,
    out_ptr,
    n_elements,
    BLOCK_SIZE: tl.constexpr,
):
    pid = tl.program_id(axis=0)
    offs = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)
    mask = offs < n_elements

    x = tl.load(x_ptr + offs, mask=mask, other=0.0)
    y = tl.load(y_ptr + offs, mask=mask, other=0.0)
    # 本块内先求和, 再原子加到标量输出 (多 program 并行)
    acc = tl.sum(x * y, axis=0)
    tl.atomic_add(out_ptr, acc)

```

Listing 5: 点积: `atomic_add` 与 `reset_to_zero`

文件: `kernel/triton/tutorial/03_matmul/step01_dot_1d.py:8--35`

对照是 `x @ w`。三维的 `w` 不是 `F.linear` 的布局, 不要混。

5.2 怎么跑

怎么跑

```

python kernel/triton/tutorial/03_matmul/step01_dot_1d.py
python kernel/triton/tutorial/03_matmul/step03_matmul_2d.py
python kernel/triton/tutorial/03_matmul/step04_matmul_3d.py

```

5.3 正确性

Note

关 `allow_tf32` 再比。fp32 下看 `max abs err`, 不要用带 `bias` 的 `F.linear` 当三维对照。

5.4 效果

Remark

教学核通常慢于 cuBLAS。课看的是分块数据流：哪一维 pid 并行、哪一维 kernel 内 for。后面 Flash 的 QK^T 就是反复调用这一套 `tl.dot`。

6 Softmax

本讲要点

- 归约维不能用 pid 切开
- Naive 整行
- Online：为 Flash 铺路
- 反向 $dx = y(dy - \sum y dy)$

配套：kernel/triton/tutorial/04_softmax/。先跑：step01_naive.py → step02_online.py。选修反向：step90_backward.py。

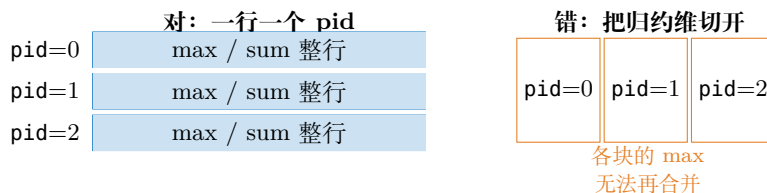
6.1 公式与数据流

对最后一维：

$$m = \max_j x_j, \quad y_j = \frac{e^{x_j - m}}{\sum_k e^{x_k - m}}.$$

归约维

归约维不能用 `program_id` 切开：要么整行装下，要么 `program` 内 `for` 分块合并统计量。



归约维留下给核内 `for`；pid 只切独立的行（或 batch）。

Naive：一个 `program` 处理若干行，`BLOCK_COLS` 覆盖整行。

```
# 只沿「行」分 program；列维一次装整行
row_pid = tl.program_id(axis=0)

offs_rows = row_pid * BLOCK_ROWS + tl.arange(0, BLOCK_ROWS)
offs_cols = tl.arange(0, BLOCK_COLS)
```

```

x_ptrs = (
    x_ptr
    + offs_rows[:, None] * x_row_stride
    + offs_cols[None, :] * x_col_stride
)
o_ptrs = (
    o_ptr
    + offs_rows[:, None] * o_row_stride
    + offs_cols[None, :] * o_col_stride
)

mask = (offs_rows[:, None] < n_rows) & (offs_cols[None, :] < n_cols)

# 越界用 -inf, 不影响行内 max
x_tile = tl.load(x_ptrs, mask=mask, other=-float("inf"))

# 每行一个 max / sum → axis=1
x_max = tl.max(x_tile, axis=1)
num = tl.exp(x_tile - x_max[:, None])
num = tl.where(mask, num, 0.0)
den = tl.sum(num, axis=1)
y = num / den[:, None]

tl.store(o_ptrs, y, mask=mask)

```

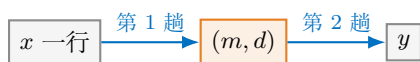
Listing 6: Naive 行 Softmax: 整行 max / sum

文件: kernel/triton/tutorial/04_softmax/step01_naive.py:46--75

Online: 沿列分块维护可合并的 (m, d) ,

$$m' = \max(m, m_{\text{blk}}), \quad d' = d e^{m-m'} + d_{\text{blk}} e^{m_{\text{blk}}-m'}.$$

两趟扫描: 先得到全局 (m, d) , 再写 $y = e^{x-m}/d$ 。这是为 N 很大, 以及后面 Flash 的在线 softmax 做准备。



$$m' = \max(m, m_{\text{blk}})$$

$$d' = d e^{m-m'} + d_{\text{blk}} e^{m_{\text{blk}}-m'}$$

online 统计量可合并, 所以 Flash 能把 K/V 留在内层循环。

```

m_i = tl.full((BLOCK_ROWS,), -float("inf"), dtype=tl.float32)
d_i = tl.zeros((BLOCK_ROWS,), dtype=tl.float32)

for start in range(0, n_cols, BLOCK_COLS):
    offs_cols = start + tl.arange(0, BLOCK_COLS)
    mask = row_mask[:, None] & (offs_cols[None, :] < n_cols)

    x_ptrs = x_ptr + offs_rows[:, None] * x_row_stride + offs_cols[None, :] * x_col_stride
    x_tile = tl.load(x_ptrs, mask = mask, other = -float("inf"))

```

```

max = tl.max(x_tile, axis = 1)
new_max = tl.maximum(max, m_i)
num = tl.exp(x_tile - new_max[:, None])

d_blk = tl.sum(num, axis = 1)
d_i = d_i * tl.exp(m_i - new_max) + d_blk
m_i = new_max

for start in range(0, n_cols, BLOCK_COLS):
    offs_cols = start + tl.arange(0, BLOCK_COLS)
    mask = row_mask[:, None] & (offs_cols[None, :] < n_cols)

    x_ptrs = x_ptr + offs_rows[:, None] * x_row_stride + offs_cols[None, :] * x_col_stride
    o_ptrs = o_ptr + offs_rows[:, None] * o_row_stride + offs_cols[None, :] * o_col_stride

    x_tile = tl.load(x_ptrs, mask = mask, other = -float("inf"))
    y = tl.exp(x_tile - m_i[:, None]) / d_i[:, None]
    tl.store(o_ptrs, y, mask = mask)

```

Listing 7: Online Softmax: 两趟列块

文件: kernel/triton/tutorial/04_softmax/step02_online.py:50--77

反向 (存 y 不存 x):

$$dx = y \odot (dy - \langle y, dy \rangle \mathbf{1}).$$

TritonSoftmax.apply 才建图; 裸 naive_softmax 没有 grad_fn。

```

y = tl.load(y_ptrs, mask = mask, other = 0.0)
dy = tl.load(dy_ptrs, mask = mask, other = 0.0)

dx = y * (dy - tl.sum(y * dy, axis = 1)[:, None])
tl.store(dx_ptrs, dx, mask = mask)

def naive_softmax_backward(y: torch.Tensor, dy: torch.Tensor) -> torch.Tensor:
    """给定 softmax 输出 y 和上游梯度 dy, 计算 dx。"""
    assert y.is_cuda and dy.is_cuda
    assert y.shape == dy.shape
    assert y.ndim in (2, 3), "入门版支持 2D/3D"

    y_contig = y.contiguous()
    dy_contig = dy.contiguous()
    dx = torch.empty_like(y_contig)

    n_cols = y_contig.shape[-1]
    n_rows = y_contig.numel() // n_cols

    BLOCK_COLS = triton.next_power_of_2(n_cols)
    assert BLOCK_COLS <= 65536, f"N={n_cols} 太大, 需要分块 Softmax backward"

    grid = lambda meta: (triton.cdiv(n_rows, meta["BLOCK_ROWS"]),)
    naive_softmax_backward_kernel[grid](
        y_contig,
        dy_contig,

```

```

        dx,
        y_contig.stride(-2),
        y_contig.stride(-1),
        dy_contig.stride(-2),
        dy_contig.stride(-1),
        dx.stride(-2),
        dx.stride(-1),
        n_rows,
        n_cols,
        BLOCK_COLS=BLOCK_COLS,
    )
    return dx

```

```

class TritonSoftmax(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x):
        y = naive_softmax(x)
        ctx.save_for_backward(y)
        return y

    @staticmethod
    def backward(ctx, dy):
        (y,) = ctx.saved_tensors
        dx = naive_softmax_backward(y, dy)
        return dx

```

Listing 8: 反向闭式与 Function

文件: kernel/triton/tutorial/04_softmax/step90_backward.py:109--160

6.2 怎么跑

怎么跑

```

python kernel/triton/tutorial/04_softmax/step01_naive.py
python kernel/triton/tutorial/04_softmax/step02_online.py
python kernel/triton/tutorial/04_softmax/step90_backward.py

```

6.3 正确性

Note

对 `F.softmax(..., dim=-1)`; 反向 `autograd.grad`。入门形状最后一维先 ≤ 1024 。

6.4 效果

Remark

完整写出 y 时, online 不一定更快 (多读一遍 x)。意义是 N 装不进寄存器, 以及给 Flash 准备 (m, ℓ) 这类统计量。

7 LayerNorm 与 RMSNorm

本讲要点

- LN vs RMS
- 行统计量
- $d\gamma/d\beta$ 用 `atomic_add`
- 推理 API 是 `rmsnorm`

配套: `kernel/triton/tutorial/05_norm/`。先跑: `step01_layernorm.py` $\rightarrow$ `step02_online_layernorm.py` $\rightarrow$ `step03_rmsnorm.py`。推理 Host API 是 `from step03_rmsnorm import rmsnorm`。LN 反向写在 `step01` 同文件。

7.1 公式与数据流

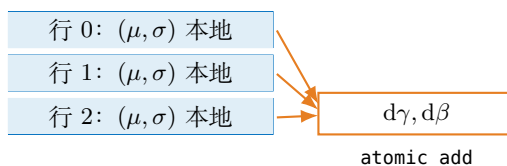
LayerNorm (最后一维 N):

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}}, \quad y = \gamma \odot \hat{x} + \beta.$$

RMSNorm 没有减均值、没有 β :

$$\text{rms} = \sqrt{\text{mean}(x^2) + \varepsilon}, \quad y = w \odot x / \text{rms}.$$

Naive LN 把整行 N 一次装下。 N 很大时会炸寄存器; online 沿 N 分块累加 $\sum x$ 、 $\sum x^2$ 。



dx 按行写; $d\gamma/d\beta$ 形状是 $(N,)$, 多行必须原子加。

```
x = tl.load(x_ptrs, mask=mask, other=0.0)
w = tl.load(w_ptrs, mask=w_mask, other=0.0)
bias = tl.load(b_ptrs, mask=w_mask, other=0.0)

# 行归约: axis=1
mean = tl.sum(x, axis=1) / N
x_center = x - mean[:, None]
# 越界位置 other=0, 不贡献 sum; 有效长度按 N (与 F.layer_norm 一致用全长)
```

```

var = tl.sum(x_center * x_center, axis=1) / N
rstd = 1.0 / tl.sqrt(var + eps)

y = x_center * rstd[:, None] * w[None, :] + bias[None, :]
tl.store(o_ptrs, y, mask=mask)

```

Listing 9: LayerNorm 前向：行均值、方差、仿射

文件：kernel/triton/tutorial/05_norm/step01_layernorm.py:69--81

反向里 dx 每行本地可写； $d\gamma, d\beta$ 形状是 $(N,)$ ，要把所有行加在一起。本块先 `sum(axis=0)`，再 `atomic_add` 到全局。autotune 必须 `reset_to_zero`。

```

# 本 tile 若干行对 (N,) 的贡献；多个 program 写同一 dw/db
dw_tile = tl.sum(tl.where(mask, dy * xhat, 0.0), axis=0)
db_tile = tl.sum(tl.where(mask, dy, 0.0), axis=0)
tl.atomic_add(dw_ptr + offs_n, dw_tile, mask=w_mask)
tl.atomic_add(db_ptr + offs_n, db_tile, mask=w_mask)

```

Listing 10: 参数梯度：tile 内求和再原子加

文件：kernel/triton/tutorial/05_norm/step01_layernorm.py:223--227

7.2 怎么跑

怎么跑

```

python kernel/triton/tutorial/05_norm/step01_layernorm.py
python kernel/triton/tutorial/05_norm/step02_online_layernorm.py
python kernel/triton/tutorial/05_norm/step03_rmsnorm.py

```

7.3 正确性

Note

LN 对 `F.layer_norm`。RMS 对 `F.rms_norm`。fp32, rtol 约 10^{-4} 。训练必须 `TritonLayerNorm.apply`，不能对裸 `layernorm(...)` 求 grad。

7.4 效果

Remark

大 N 时分块版接近 torch，是因为 naive 整行会炸寄存器，不是因为公式更快。教学 LN 大 hidden 仍打不过 `F.layer_norm`（背后是 cuDNN），训练默认不换它。

8 注意力：从 Naive 到 Flash

本讲要点

- Naive / MHA 是 Host 串
- 单头 Flash：论文双循环
- 教学多头 causal：grid=(B,H)
- 加速版：Q 块并行与 GQA
- RoPE / GQA / MLA

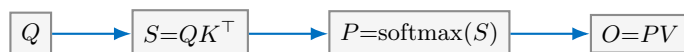
配套：kernel/triton/tutorial/06_attention/。先跑：naive → MHA → 单头 Flash → **教学多头 causal** → RoPE → GQA。加速版 step04_flash_causal_mha.py 给训练用；对照课请先读教学版。MLA 选修。Qwen3.5 的 DeltaNet 本课不实现。

8.1 Naive 与多头：Host 串起来不算融合

单头：

$$S = \frac{QK^\top}{\sqrt{d}}, \quad P = \text{softmax}(S), \quad O = PV.$$

S, P 是 (S, S) ，写进 HBM。多头把 (B, H, S, d) 展成 (BH, S, d) 再调三维 matmul。这是 Host 上多次 launch，**不是** kernel 融合。



Naive: S, P 是 (S, S) ，写进 HBM。多头若 Host 串 launch，不是融合。

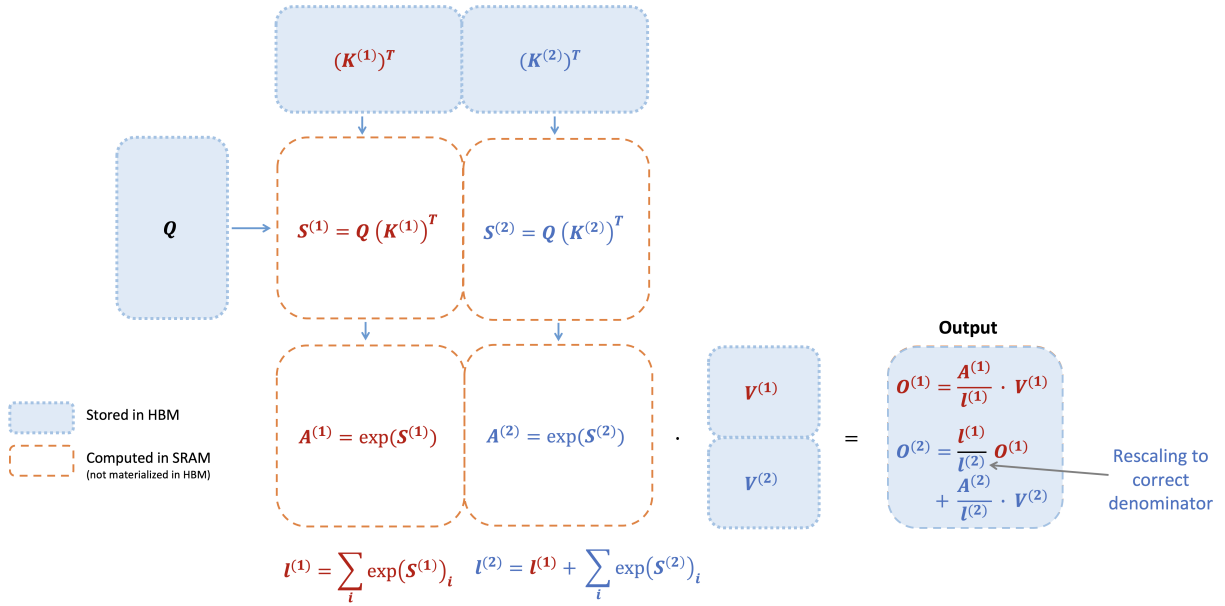
对照

对照用 `F.scaled_dot_product_attention(is_causal=True)`。不要用带投影的 `multi_head_attention_forward`。SDPA 会选后端，不等于「一次 Flash」。

8.2 单头 Flash：论文双循环

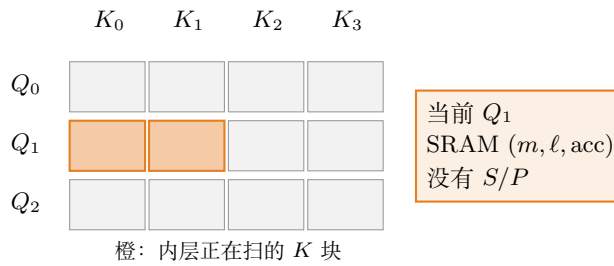
数据流以 FlashAttention-2 论文为准 (Dao. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. ICLR 2024; <https://arxiv.org/abs/2307.08691>。仓库根目录有同名 PDF)。要点：外层 Q 行块、内层 K/V 列块，片上维护 (m, ℓ, acc) ，不把 S/P 写回 HBM。外层可以映射成 `program_id`，内层**必须** `for`——online 统计量不能按列拆成不同 program。

论文 Fig. 1 把「切 K/V 、片上 softmax、用 ℓ 重标定 O 」画在一张图里。蓝实线框在 HBM，橙虚线框在 SRAM（不物化）：



论文 Fig. 1: Dao. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. ICLR 2024。
 K, V 切成两块时，先算 $(S^{(1)}, A^{(1)}, O^{(1)})$ ，再与第二块用 ℓ 重标定得到正确的 O 。

课上把同一件事收成「谁是 pid、谁必须核内 for」:



Flash 双循环: 外 Q 可映射成 pid, 内 K/V 必须在同一 program 里扫。

论文 Algorithm 1 就是这层双循环。外 i 扫 Q 块 (我们的 BLOCK_M), 内 j 扫 K/V 块 (BLOCK_N), 片上更新 m_i, ℓ_i, O_i , 最后写 O_i 和 $\text{LSE}_i = m_i + \log \ell_i$:

Algorithm 1 FLASHATTENTION-2 forward pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, block sizes B_c, B_r .

- 1: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
 - 2: Divide the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, and divide the logsumexp L into T_r blocks $L_1, \dots, L_{T_r}$ of size B_r each.
 - 3: **for** $1 \leq i \leq T_r$ **do**
 - 4: Load $\mathbf{Q}_i$ from HBM to on-chip SRAM.
 - 5: On chip, initialize $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}, \ell_i^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}, m_i^{(0)} = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$.
 - 6: **for** $1 \leq j \leq T_c$ **do**
 - 7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 8: On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 9: On chip, compute $m_i^{(j)} = \max(m_i^{(j-1)}, \text{rowmax}(\mathbf{S}_i^{(j)})) \in \mathbb{R}^{B_r}, \tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\ell_i^{(j)} = e^{m_i^{(j-1)} - m_i^{(j)}} \ell_i^{(j-1)} + \text{rowsum}(\tilde{\mathbf{P}}_i^{(j)}) \in \mathbb{R}^{B_r}$.
 - 10: On chip, compute $\mathbf{O}_i^{(j)} = \text{diag}(e^{m_i^{(j-1)} - m_i^{(j)}}) \mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$.
 - 11: **end for**
 - 12: On chip, compute $\mathbf{O}_i = \text{diag}(\ell_i^{(T_c)})^{-1} \mathbf{O}_i^{(T_c)}$.
 - 13: On chip, compute $L_i = m_i^{(T_c)} + \log(\ell_i^{(T_c)})$.
 - 14: Write $\mathbf{O}_i$ to HBM as the i -th block of $\mathbf{O}$.
 - 15: Write L_i to HBM as the i -th block of L .
 - 16: **end for**
 - 17: Return the output $\mathbf{O}$ and the logsumexp L .
-

论文 Algorithm 1: FlashAttention-2 前向。教学单头把两层 **for** 都留在核里，方便一行行对照。

教学单头 `grid=(B,)`，双重 **for** 都在核内：

```
# 外循环: Q 行块 i
for start_m in range(0, S, BLOCK_M):
    offs_m = start_m + tl.arange(0, BLOCK_M)
    q_mask = (offs_m[:, None] < S) & (offs_d[None, :] < D)

    q = tl.load(
        q_batch + offs_m[:, None] * stride_qm + offs_d[None, :] * stride_qd,
        mask=q_mask,
        other=0.0,
    )

    m_i = tl.full((BLOCK_M,), -float("inf"), dtype=tl.float32)
    l_i = tl.zeros((BLOCK_M,), dtype=tl.float32)
    acc = tl.zeros((BLOCK_M, BLOCK_D), dtype=tl.float32)

    # 内循环: K/V 列块 j (不能拆成 program_id, 否则 (m,l) 无法在线合并)
    for start_n in range(0, S, BLOCK_N):
        offs_n = start_n + tl.arange(0, BLOCK_N)
        kv_mask = (offs_n[:, None] < S) & (offs_d[None, :] < D)

        k = tl.load(
            k_batch + offs_n[:, None] * stride_km + offs_d[None, :] * stride_kd,
            mask=kv_mask,
            other=0.0,
        )
        v = tl.load(
            v_batch + offs_n[:, None] * stride_vm + offs_d[None, :] * stride_vd,
            mask=kv_mask,
```

```

        other=0.0,
    )

    qk = tl.dot(q, tl.trans(k)) * scale

```

Listing 11: 单头 Flash: 外 Q 内 KV

文件: kernel/triton/tutorial/06_attention/step03_flash_single.py:83--114

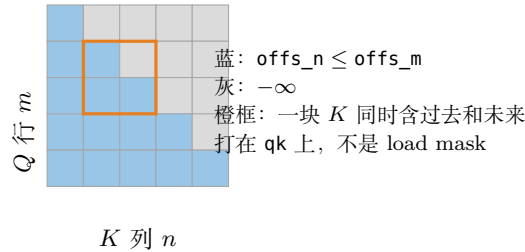
shared memory

Head dim D 保持 64/128。把 hidden=512 当一头会让 (BLOCK_M, D) 爆 shared memory。本机 smem 上限大约 101376 字节；教学核把整段 D 放进 SRAM, $D=512$ 会跑不了。

8.3 教学多头 causal

文件: step04_flash_causal_mha_teach.py。

grid=(B,H), 一个 program 一个头, 内部仍是论文双循环。Causal 打在 qk 上: $\text{offs}_n \leq \text{offs}_m$ 才保留, 否则 $-\infty$ 。这不是 tl.load 的越界 mask; K/V 块里常常同时含过去和未来。



一块 K 里常常同时有过去和未来, 不能靠越界 mask 代替 causal。

正向必须写下

$$\text{LSE}_i = m_i + \log \ell_i,$$

形状 (B, H, S) , 反向用 $P_{ij} = \exp(S_{ij} - \text{LSE}_i)$ 重算, 不能存完整 P 。

反向教学布局: 外层 KV 列块 (本地攒 dK/dV), 内层 Q 行块 (dQ 用 atomic_add)。因此 Host 上 dQ 必须 zeros_like, 不能 empty。这就是论文 Fig. 2 右边: 每个 worker 负责注意力矩阵的一列块。

```

pid_b = tl.program_id(axis=0)
pid_h = tl.program_id(axis=1)
offs_d = tl.arange(0, BLOCK_D)

q_bh = q_ptr + pid_b * stride_qb + pid_h * stride_qh
k_bh = k_ptr + pid_b * stride_kb + pid_h * stride_kh
v_bh = v_ptr + pid_b * stride_vb + pid_h * stride_vh
o_bh = o_ptr + pid_b * stride_ob + pid_h * stride_oh
lse_bh = lse_ptr + pid_b * stride_lb + pid_h * stride_lh

for start_m in range(0, S, BLOCK_M):
    offs_m = start_m + tl.arange(0, BLOCK_M)

```

```

q_mask = (offs_m[:, None] < S) & (offs_d[None, :] < D)

q = tl.load(
    q_bh + offs_m[:, None] * stride_qm + offs_d[None, :] * stride_qd,
    mask=q_mask,
    other=0.0,
)

m_i = tl.full((BLOCK_M,), -float("inf"), dtype=tl.float32)
l_i = tl.zeros((BLOCK_M,), dtype=tl.float32)

```

Listing 12: 教学 FA: grid 是 (B,H), 外层 for 扫 Q 块

文件: kernel/triton/tutorial/06_attention/step04_flash_causal_mha_teach.py:70--91

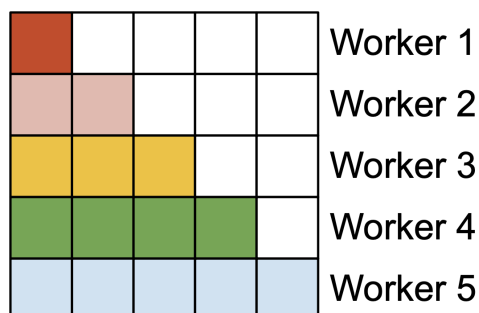
Note

数值对照必须走 TritonFlashAttentionCausalMHA.apply, 不能对裸 Triton 的 O 求 grad。一次 `grad(o, (q,k,v), do)` 求出三个梯度。

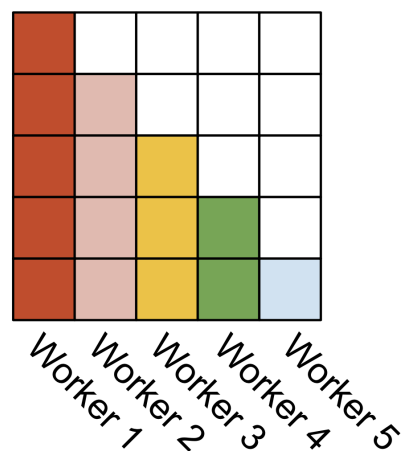
8.4 加速版: 训练在用的核

文件: `step04_flash_causal_mha.py`。相对教学版, 核心是把论文 Fig. 2 的并行做进去: 前向每个 worker 一行块 (一块 Q), 反向每个 worker 一列块 (一块 K/V , dQ 原子加)。

Forward pass

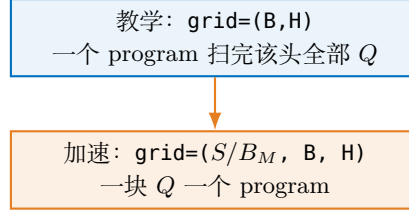


Backward pass



论文 Fig. 2: Dao. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. ICLR 2024.

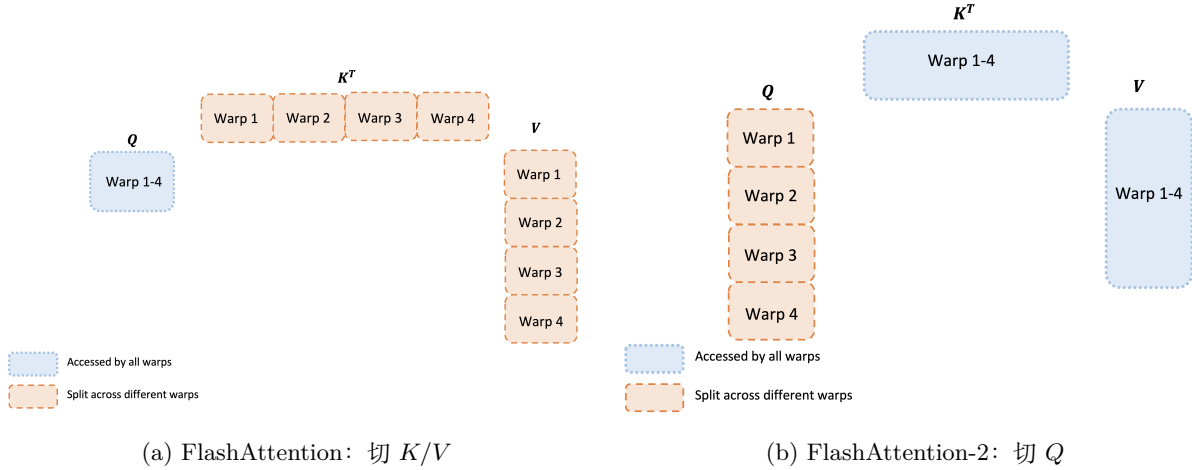
左: 前向按行块并行 (加速版的 Q 块 pid)。右: 反向按列块并行 (教学反向的 KV 外层)。



先读懂教学版的双循环，再看 Q 块并行和 GQA 的 GROUP。

1. Q 块并行: $\text{grid}=(\text{cdiv}(S, \text{BLOCK_M}), B, H)$ ，每个 program 一块 Q。
2. GQA: $\text{GROUP}=\text{H_q}/\text{H_kv}$, $\text{pid_hkv} = \text{pid_h} // \text{GROUP}$ ，不再 repeat_interleave KV。
3. Causal 用循环上界收掉未来 KV 块 (Triton 不能 break)。
4. 反向改成 Q 并行: 本地写 dQ ，对 dK/dV atomic_add (fp32 累加)。

论文 Fig. 3 是同一个 thread block 里 warp 怎么切。FlashAttention 切 K/V (split-K, 中间结果要进 shared memory 再加); FlashAttention-2 改切 Q ，各 warp 自己写出对应的 O 片，少一次 smem 同步。Triton 的 `tl.dot` 不让你直接排 warp，但这解释了官方 FA2 为什么比 FA1 快。



论文 Fig. 3: Dao. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. ICLR 2024.

蓝: 所有 warp 共享; 橙: 切到不同 warp。

```

):
    pid_m = tl.program_id(axis=0)
    pid_b = tl.program_id(axis=1)
    pid_h = tl.program_id(axis=2)
    pid_hkv = pid_h // GROUP
    start_m = pid_m * BLOCK_M
    offs_d = tl.arange(0, BLOCK_D)
    offs_m = start_m + tl.arange(0, BLOCK_M)

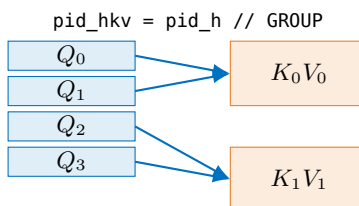
    q_bh = q_ptr + pid_b * stride_qb + pid_h * stride_qh
    k_bh = k_ptr + pid_b * stride_kb + pid_hkv * stride_kh
    v_bh = v_ptr + pid_b * stride_vb + pid_hkv * stride_vh
    o_bh = o_ptr + pid_b * stride_ob + pid_h * stride_oh

```

```
lse_bh = lse_ptr + pid_b * stride_lb + pid_h * stride_lh
```

Listing 13: 加速 FA: Q 块并行与 GQA GROUP

文件: kernel/triton/tutorial/06_attention/step04_flash_causal_mha.py:67--80



GQA: $H_q > H_{kv}$ 。核里除法取 KV 头, 不要 repeat_interleave。

llm_train 走加速版。先把教学版的数据流搞懂, 再看这些并行与 GQA 改动。

8.5 RoPE、GQA、MLA

RoPE 在 step05_rope.py, 按 head dim 旋转。GQA 是 $H_q > H_{kv}$: 推理可用 step06_gqa.py 的前向 flash_attention_gqa; 训练必须 flash_attention_gqa_train (有图)。MLA 在 step07_mla.py, 教学版吸收仍偏 Host。

8.6 怎么跑

怎么跑

```
python kernel/triton/tutorial/06_attention/step01_naive.py
python kernel/triton/tutorial/06_attention/step02_mha.py
python kernel/triton/tutorial/06_attention/step03_flash_single.py
python kernel/triton/tutorial/06_attention/step04_flash_causal_mha_teach.py
python kernel/triton/tutorial/06_attention/step04_flash_causal_mha.py
python kernel/triton/tutorial/06_attention/step05_rope.py
python kernel/triton/tutorial/06_attention/step06_gqa.py
```

8.7 正确性

Note

fwd 教学默认 fp32, rtol = 10^{-2} 。lse 是 (B, H, S) , 不要和 4D 的 O 比 shape。反向 smem 比正向紧: 同时驻留 Q, K, V, O, dO 和 tile 上的 P, dS 。64 × 64 × D 在本机曾 OutOfResources, 教学反向默认 32 × 32。

8.8 效果

Remark

Flash 省的是 HBM 上的 S/P , FLOPs 并不更少。墙钟是否赢 SDPA 取决于形状和是否走库函数 Flash。看 launch 次数去做下一章。

9 用 nsys 看 Launch

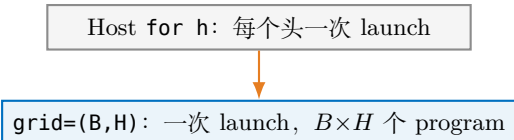
本讲要点

- launch = 一次 kernel[grid]
- 看名字和次数
- 教学 FA 的 D 太大炸 smem
- 脚本: run_nsys.sh

配套: kernel/triton/tutorial/07_profiling/. 依赖: 先做完第 8 章 Flash。本机 torch.profiler 的 CUPTI 常常采不到 GPU, 主线是 nsys。ncu 本课不做。

9.1 看什么

一次 kernel[grid](...) 就是一次 launch。多头不要 Python for h 调 H 次。nsys 首先看 **kernel 名字和次数**, 不要只盯总微秒是否赢 SDPA。



nsys 先数 kernel 名字和次数。Python 循环调核, 时间线会被切碎。

对照三种实现: PyTorch、naive attention、教学 Flash。形状先用默认 $D = 64$ 。教学 Flash 把整段 D 放进 SRAM, $D = 512$ 会因 smem 跑不了, 那不是 nsys 坏了。

9.2 怎么跑

怎么跑

```
source .venv/bin/activate
bash kernel/triton/tutorial/07_profiling/run_nsys.sh
```

或手动:

手动 nsys

```
cd kernel/triton/tutorial/07_profiling
nsys profile --stats=true --force-overwrite=true \
  --trace=cuda,nvtx,osrt \
  -o nsys_out/flash \
  python nsys_targets.py --impl flash
```

--impl: torch / naive / flash。脚本结束会写 nsys_out/compare.md。

9.3 正确性

Note

profiling 不替代数值对照。先保证第 8 章 allclose，再采时间。换形状时三种实现用同一 B, S, D 。

9.4 效果

Remark

常见结论（以你机器打印为准）：naive 多次 GEMM + softmax，launch 多、中间张量大；Flash 把内层留在一个 kernel。SDPA 可能走完全不同的后端，总时间更短并不矛盾。

10 推理实战：接到 Qwen3.5

本讲要点

- 0.8B 混合架构
- 只换 RMS / SwiGLU / GQA
- 裸 kernel，无计算图
- 和 vLLM 比的是调度

配套：projects/qwen35_infer/。一条 generate：prefill 整段 prompt，decode 时维护 KV。不要在这里写迷你 vLLM。

10.1 公式与数据流

Qwen/Qwen3.5-0.8B 不是纯 Transformer：24 层大致是 $6 \times (3 \times (\text{Gated DeltaNet} \rightarrow \text{FFN}) + 1 \times (\text{Gated Attention} \rightarrow \text{FFN}))$ ，大约 18 层线性注意力 + 6 层 softmax 注意力（GQA，8 个 Q 头 / 2 个 KV 头，head_dim = 256）。第一版 DeltaNet 仍用 PyTorch（玩具模型里用一层 Linear 占位）。

本课只换三处前向：

- RMSNorm: rmsnorm

- SwiGLU MLP: swiglu_pro 再 @ W_down
- GQA Flash: flash_attention_gqa, RoPE 用 apply_rope_offset

```
def swiglu_mlp(x: torch.Tensor, w_gate: torch.Tensor, w_up: torch.Tensor, w_down: torch.Tensor)
    -> torch.Tensor:
    """out = (silu(x@w_gate) * (x@w_up)) @ W_down。x:(B,S,D)。"""
    orig = x.shape
    x3 = x.reshape(orig[0], -1, orig[-1]).contiguous() if x.ndim == 3 else x.unsqueeze(0)
    fused = swiglu_pro(x3, w_gate, w_up)
    y = fused @ w_down
    return y.reshape(orig) if x.ndim == 3 else y.squeeze(0)
```

Listing 14: 推理侧把课上的前向 API 串起来

文件: projects/qwen35_infer/kernels.py:14--20

没有图

这些是裸 **Triton 前向**，没有 autograd 图，不能拿去训练。8GB 不要一上来 bf16 4B/9B；0.8B 短上下文够用。官方 256K 上下文本课不跑。

10.2 怎么跑

基础环境见仓库根 README。本目录额外可装 transformers；vLLM 体积大，WSL 上可能难装，不是验收必选项。

怎么跑

```
python projects/qwen35_infer/smoke_test.py
python projects/qwen35_infer/generate.py --backend toy --kernels eager
python projects/qwen35_infer/generate.py --backend toy --kernels triton
python projects/qwen35_infer/bench.py --backend-list toy_eager,toy_triton
```

有权重时再加 HuggingFace / vLLM 对照，短上下文（256–1024）。

10.3 正确性

Note

同一权重、同一组短 prompt、greedy、相同 max_new_tokens。先看文本 / token 是否一致，再看 TTFT、decode tok/s、峰值显存。玩具模型不下载权重也能做核对齐。

10.4 效果

Remark

vLLM 通常更快，因为 PagedAttention、continuous batching、CUDA Graph 和调度，不是某一个 Softmax 核。课的结论应是：换核后这一层相对 eager 快了多少，端到端还慢在哪。

11 训练对照：Function 与三后端

本讲要点

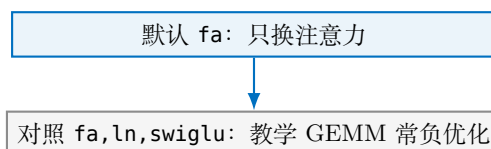
- 必须 `Function.apply`
- 默认只换 FA
- Adam 不被 autocast 减掉
- `compile` 常赢端到端

配套：projects/llm_train/。架构是 **LayerNorm + RoPE + GQA + SwiGLU**（训练骨架用 LN，不是 RMSNorm），不是 Qwen3.5 的 DeltaNet。只跑若干 step：看 step 时间、tok/s、峰值显存、loss 是否同量级，不训到可用 chatbot。

11.1 公式与数据流

推理可以调裸前向；训练必须 `torch.autograd.Function`。 `Function.apply` 才建图，自己调 `forward` 不会进 `backward`。 `save_for_backward` 只存反向真正用到的量：Softmax 存 y 不存 x ；Flash 存 Q, K, V, O, LSE ，不能存完整 P 。

默认 `--triton-ops fa`：只换 FlashAttention，LN 与 MLP 仍走 PyTorch / cuBLAS。



训练图里能换核，不等于该把所有层都换成教学实现。

```
hidden -> F.layer_norm -> FA Function (GROUP) -> residual
      -> F.layer_norm -> silu(x@Wg)*(x@Wu) + Linear down -> residual
```

教学对照再开 `--triton-ops fa,ln,swiglu`。MLP 的 down 始终 `nn.Linear`。教学大 GEMM 打不过 cuBLAS，开了会负优化。

```
def set_triton_ops(spec: str) -> None:
    """triton 后端默认只换 FA。大 GEMM 的教学 SwiGLU/LN 会负优化，需要时再开。"""
    global _TRITON_EXTRA
    parts = {p.strip() for p in spec.split(",") if p.strip()}
    unknown = parts - {"fa", "ln", "swiglu"}
    if unknown:
        raise ValueError(f"未知 triton op {unknown}, 只能是 fa,ln,swiglu")
```

```
_TRITON_EXTRA = parts - {"fa"}
```

```
class LayerNorm(nn.Module):
    """backend=triton 且 --triton-ops 含 ln 时走 TritonLayerNorm; 否则 F.layer_norm."""

    def __init__(self, dim: int, eps: float = 1e-5, backend: str = "eager"):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(dim))
        self.bias = nn.Parameter(torch.zeros(dim))
        self.eps = eps
        self.backend = backend

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        if self.backend == "triton" and "ln" in _TRITON_EXTRA:
            return TritonLayerNorm.apply(x, self.weight, self.bias, self.eps)
        return F.layer_norm(x, (x.shape[-1],), self.weight, self.bias, self.eps)
```

Listing 15: 训练默认只换 FA; LN / SwiGLU 需显式打开

文件: projects/llm_train/kernels.py:31--54

11.1.1 显存账

Adam 不在 autocast 里

混合精度 (autocast bf16) 减的是激活和 matmul, 不是 Adam 状态。约 1B 参数全参 AdamW: 权重 bf16 约 2 GB, Adam m/v 常仍是 fp32 约 8 GB, 再加梯度与激活。8GB 笔记本全参 1B 会 OOM。默认 --config small (约 0.25B, seq = 256); 1b 失败时看脚本打印的账即可, 不算课程失败。

11.2 怎么跑

怎么跑

```
python projects/llm_train/train.py --config small --backend eager --steps 5
python projects/llm_train/speed.py --config small --steps 20
python projects/llm_train/bench.py --config small --steps 10
```

Triton tile 在 projects/llm_train/tiles.py, 或命令行 --fa-block-m 等覆盖。显式 launch 必须带 num_stages, 否则 JIT 默认 3 级流水会把 smem 乘过本机上限。FA 反向不要上 64×64 。

11.3 正确性

Note

loss 同量级即可，不要求 bit 级一致。裸 kernel 没有 `grad_fn`；autocast 下 QKV 常是 bf16、交叉熵回传的 dO 常是 fp32，`tl.dot` 要求两侧同 dtype，加速 FA 会把 dO cast 到 Q 的 dtype。

11.4 效果

三后端：

表 4：训练对照里实际在跑什么

后端	实现
eager	<code>F.layer_norm</code> + <code>SDPA(enable_gqa)</code> + silu GEMM
compile	同一套 eager + <code>torch.compile</code>
triton (默认 fa)	只换 FA；LN / MLP 仍 cuBLAS
fa,ln,swiglu	再换教学 LN / 融合 SwiGLU

Conclusion

`compile` 常赢端到端。Triton 默认只比 FA 对 SDPA；不要用教学 Linear 去和 cuBLAS 比整步时间。数字以你机器 `speed.py` 为准。

A 容易踩的坑

下面从课上反复出现的问题里抽出仍有效的条目，不按作者进度板逐条抄。

课上反复出现的坑

- (a) 归约维（最后一维）不能随意用 `program_id` 切开；要么整行装下，要么 `program` 内 `for` 合并统计量。
- (b) Online Softmax / 分块 Norm 往往是为了 N 很大或为 Flash 铺路；单独写出完整 Softmax 时 online 不一定更快。
- (c) Naive 整行 `BLOCK_N = next_power_of_2(N)` 在 $N = 8192$ 时会寄存器爆炸。
- (d) 对照要公平：TF32、`synchronize`、`warmup`。
- (e) `@triton.jit` 里不能调 Python wrapper；组合算子在 Host 串调用。Host 串多次 launch 不算 kernel 融合。
- (f) 3D matmul 接多头：(B, H, S, d) 先展成 (BH, S, d)。Flash 多头则 `grid=(B,H)`，基址加 `stride_h`。
- (g) `atomic_add` + autotune 必须 `reset_to_zero`。
- (h) SDPA $\neq$ 单一 Flash：对照用 `scaled_dot_product_attention`。
- (i) FA2 是双循环：外 Q 、内 K/V 。内层必须 `for`。
- (j) Launch = 一次 `kernel[grid](...)`。不要 Python `for h`。
- (k) Causal 加在 `qk` 上 (`offs_n <= offs_m`)，不是 load 的越界 mask。Triton 没有 `break`。
- (l) Head dim 用 64/128。教学核 D 太大炸 smem（本机上限约 101376）。
- (m) 有 `@triton.autotune` 就不要再传 `BLOCK_*`；`grid` 用 `lambda meta`。转置是 `tl.trans` 不是 `tl.transpose`。
- (n) Triton kernel 不进计算图，必须 `Function`。`apply` 才建图。
- (o) `save_for_backward` 只存反向要用的量。Flash 不能存完整 P 。
- (p) 裸输出没有 `grad_fn`。对照用 `Function.apply` vs SDPA，一次求出 dQ, dK, dV 。
- (q) `lse` 是 (B, H, S)。`dQ` atomic 必须从零开始。
- (r) FA 反向 smem 更紧；教学反向 32×32 。显式 launch 记得 `num_stages`。
- (s) 跑示例用 `python path.py`，不要 `bash` 直接执行没有 shebang 的 `.py`。
- (t) 教学 GEMM / LN 打不过 cuBLAS：训练默认只换 FA。

B 符号与文件

表 5: 符号

符号	含义
B, H, S, D	batch、头数、序列、head dim
H_q, H_{kv}	GQA 的 Q 头与 KV 头
m, ℓ / LSE	在线 softmax 的行最大值与归一化； $LSE = m + \log \ell$
BLOCK_M/N/K	输出行块 / 列块 / 归约块
program / pid	一次并行实例，接近 CUDA block
SM / warp	GPU 上的计算核心；32 线程锁步调度
HBM / SRAM	全局内存 vs SM 上的 shared memory

表 6: 仓库入口

路径	内容
kernel/triton/tutorial/	01-07 算子
.../step04_flash_causal_mha_teach.py	教学 FA
.../step04_flash_causal_mha.py	加速 FA (训练)
projects/qwen35_infer/	推理 generate
projects/llm_train/	训练对照
docs/textbook/	本书
ICLR-2024-flashattention-2-...pdf	FA2 论文 (第 8 章插图出处)

参考文献

Dao. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. ICLR 2024. <https://arxiv.org/abs/2307.08691>。第 8 章 Fig. 1-3 与 Algorithm 1 直接引自该文，用于对照仓库里的教学 / 加速核。